Robotics and Its Application AI352

Unit 6: Robot Planning

Topics Cover: Path Planning: Search Algorithms, sampling based methods, Motion planning: Trajectory generation, obstacle avoidance, Task planning: Hierarchical planning, task decomposition

By Dr. Aaishwarya Bajaj

Teaching Assistant DoAI, SVNIT,Surat

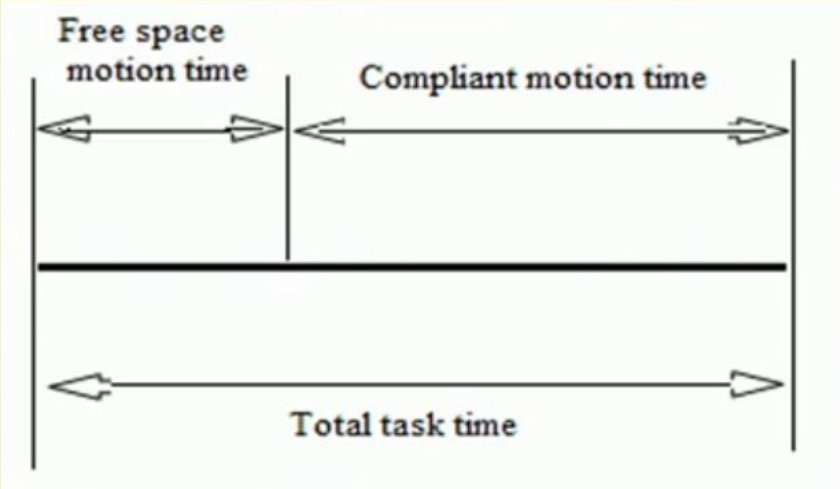
Robot Planning

The Hierarchy of Autonomy

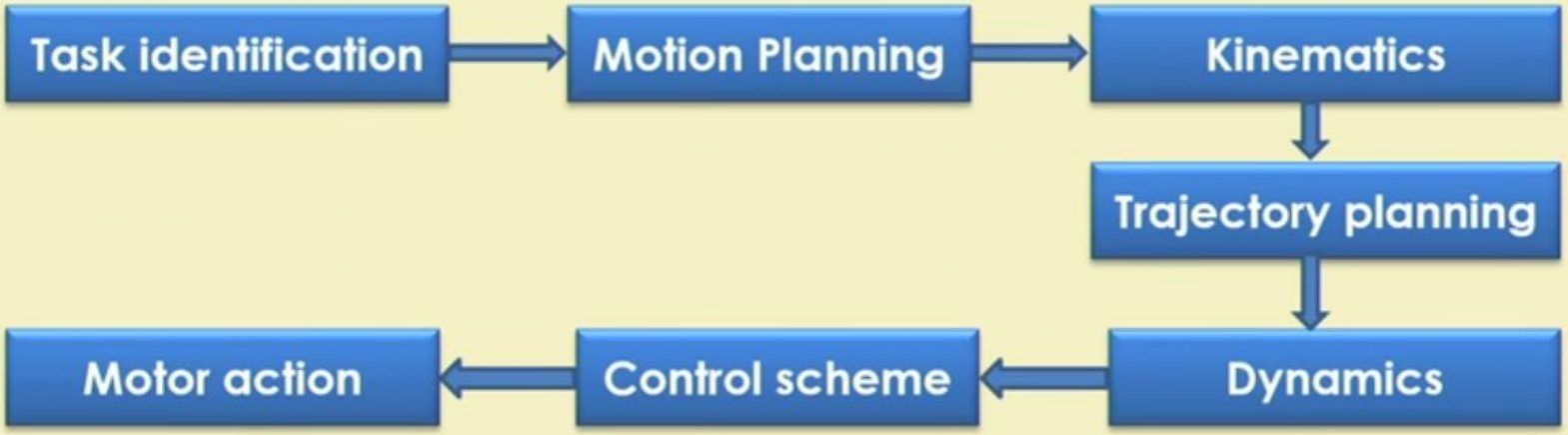
- **Definition:** Robot planning is the process of generating a strategy to achieve a goal while respecting physical and environmental constraints.
- The Three Pillars:
- Task Planning: Strategic "What to do" (Logical sequencing).
- Path Planning: Tactical "Where to go" (Geometry and obstacles).
- Motion Planning: Operational "How to move" (Dynamics and trajectories).

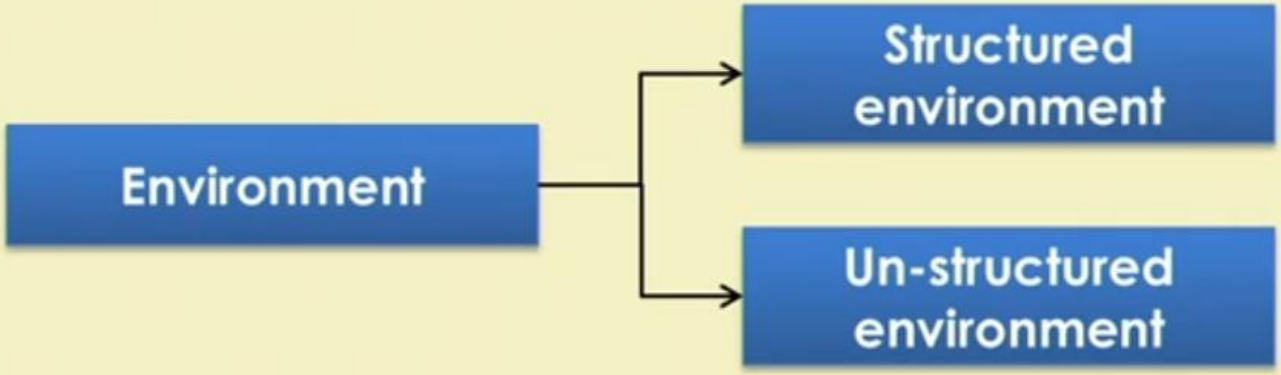
Path Planning

- To determine the course of action/path while moving from an initial position to a final position.



Sequence of Robotic Action





Motion Planning Approaches

Global Approach/ Act-After-Thinking process/ Off-line planning (motion planning with complete information)

Local Approach/ Act-While-Thinking process/ On-line planning (motion planning with incomplete information)

Motion Planning Schemes

Traditional schemes/ Algorithmic Approaches

Graph-based methods

(i) Visibility graph
(ii) Voronoi diagram
(iii) Cell decomposition
(iv) Tangent graph
(v) Accessibility graph

Analytical Approaches

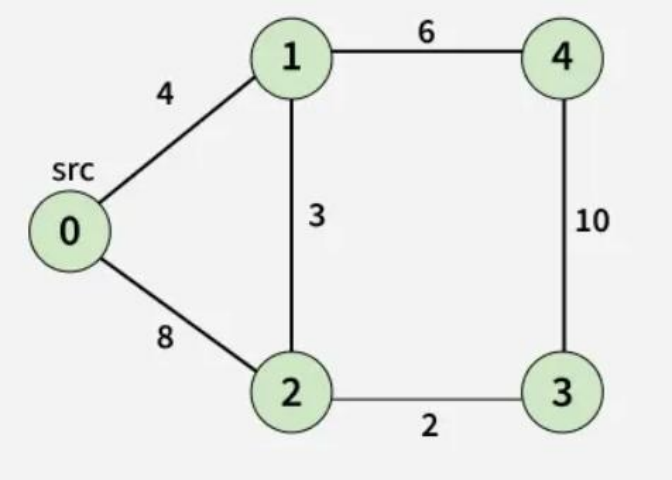
(i) Potential Field Approach
(ii) Path Velocity Decomposition
(iii) Incremental Planning
(iv) Probabilistic Approach
(v) Relative Velocity Approach
(vi) Reactive Control Strategies (Behavior-Based Robotics)

Non-Traditional schemes (using soft computing)

(i) Fuzzy logic – based
(ii) Neural networks – based approaches

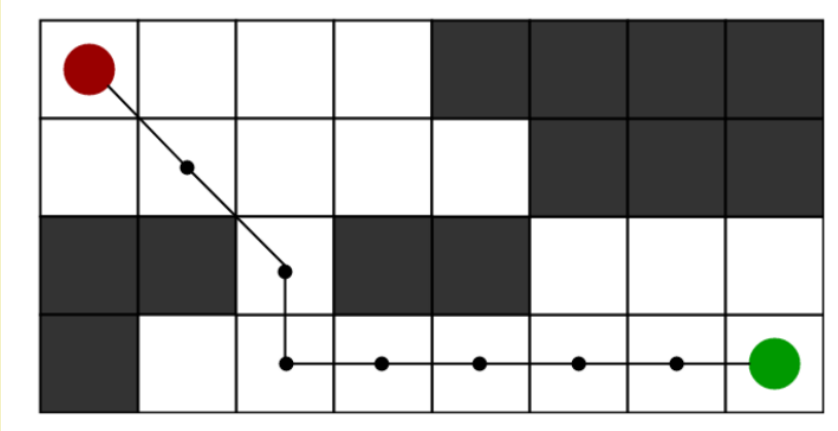
Search Algorithms (Dijkstra's)

- Edsger W. Dijkstra, a Dutch computer scientist, conceived this algorithm in 1956 and published it in 1959. Interestingly, he designed it in about 20 minutes while sitting at a cafe with his fiancée, aiming to demonstrate the capabilities of a new computer called the ARMAC. He wanted a way to find the shortest path between two cities in the Netherlands.
- In robotics, Dijkstra is less about getting from A to B as fast as possible and more about global coverage.
- Example: A robotic vacuum cleaner or a search-and-rescue robot that needs to map an entire unknown area. Because Dijkstra explores all directions, it guarantees you find the shortest path to every point, which is useful when the robot needs to explore a whole room.



Search Algorithms (A*)

- A* was developed at the Stanford Research Institute (SRI) in 1968 by Peter Hart, Nils Nilsson, and Bertram Raphael. They were working on Shakey the Robot, the first general-purpose mobile robot capable of reasoning about its actions. They needed a way to make Shakey move more efficiently than Dijkstra's allowed
- To approximate the shortest path in real-life situations, like- in maps, games where there can be many hindrances.
- We can consider a 2D Grid having several obstacles and we start from a source cell (colored red below) to reach towards a goal cell (colored green below)



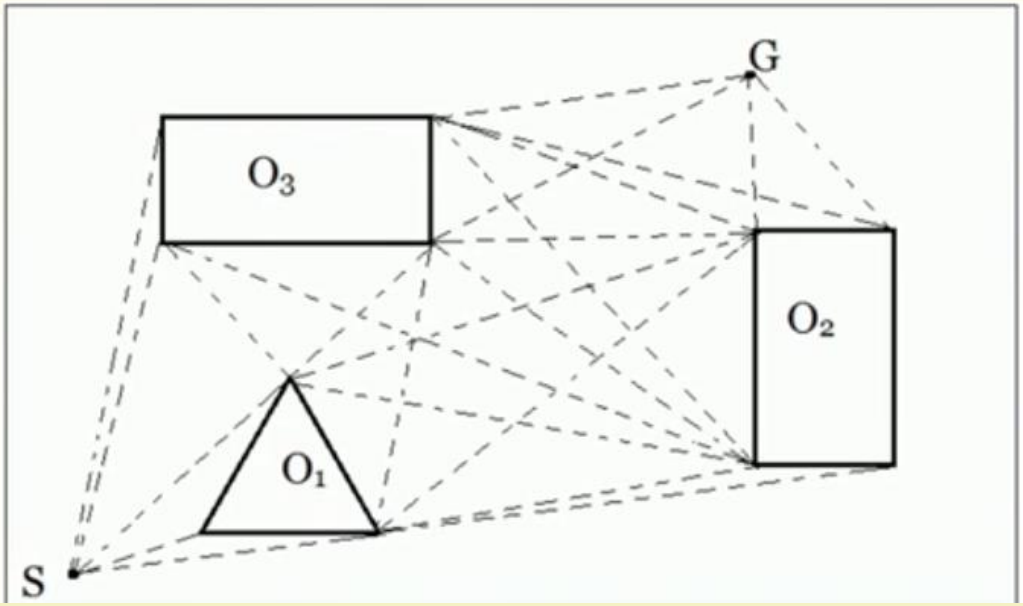
- $F(n)=g(n)+h(n)$

g(n): The actual cost from the **start** to the current node.

h(n): The estimated cost from the current node to the **goal** (the heuristic).

Visibility Graph (Nilsson 1969)

- It connects those vertices of obstacles, which are visible from one another.

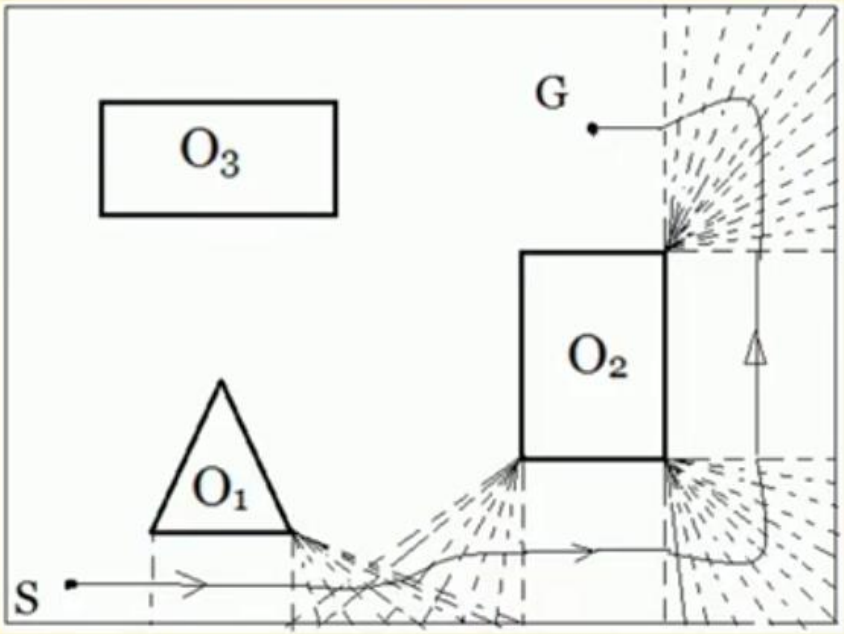


Assumption's:
1. Point Robots
2. Obstacles are fixed

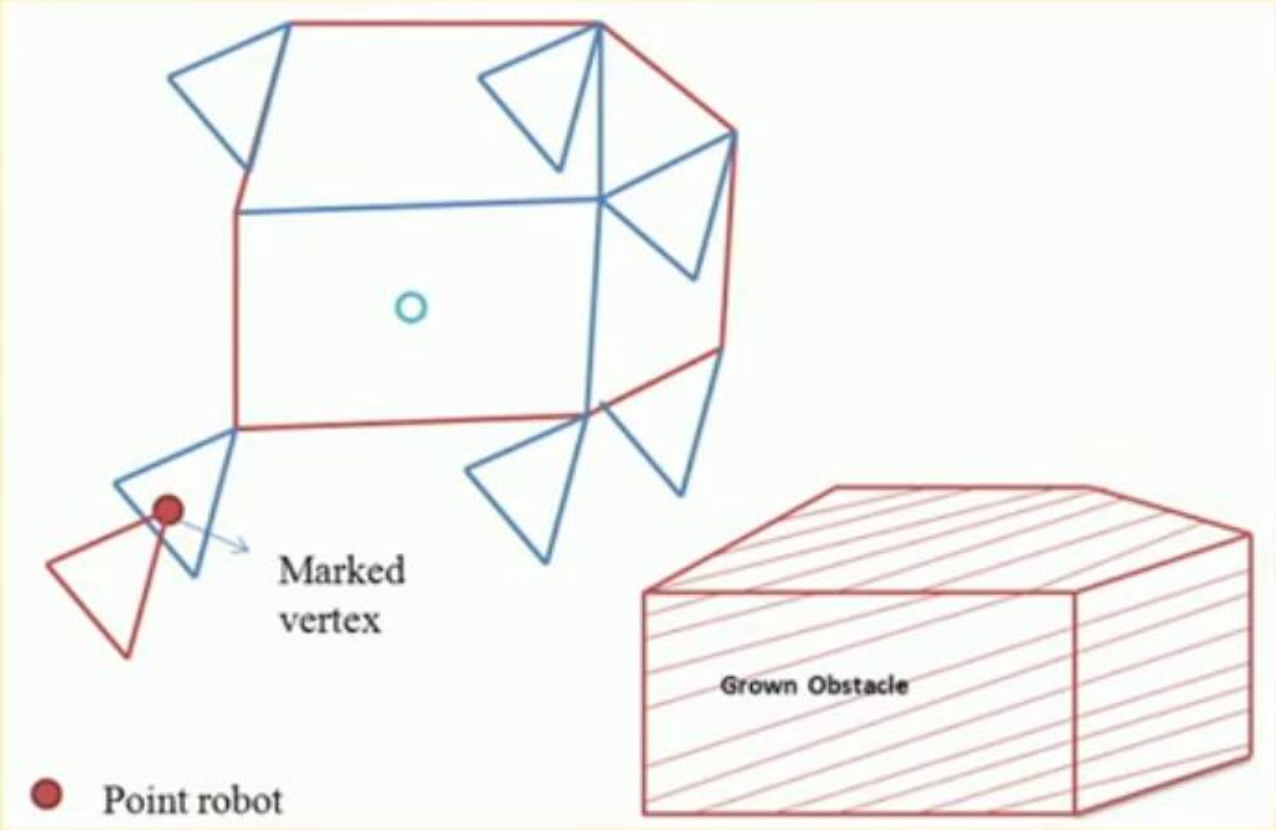
Find Path Problem

Voronoi Diagram (Dunlaing et al., 1986)

- Represents the locus of points those are equidistant from at least two of the boundaries (obstacle).

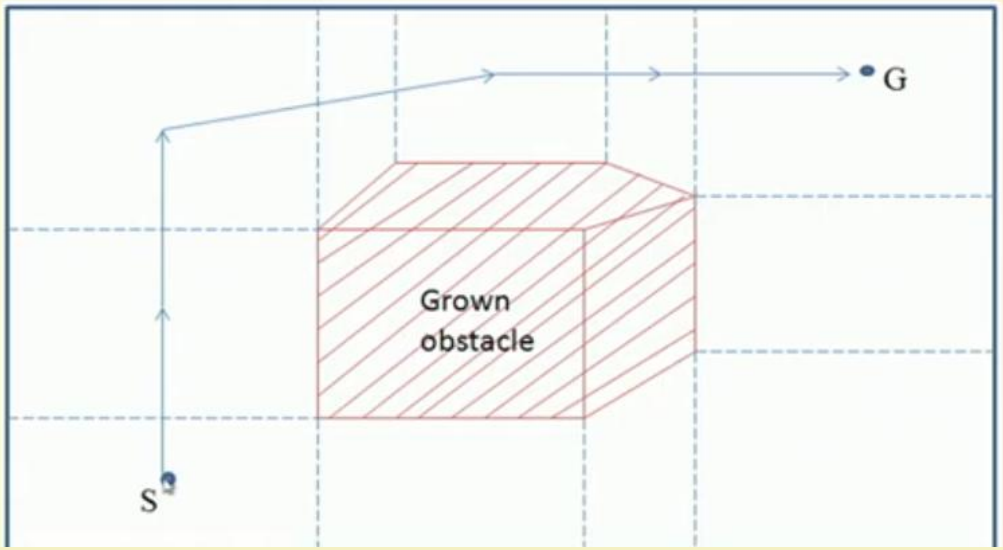


Cell Decomposition (Lozano Perez, 1983)
Configuration space (C-space)



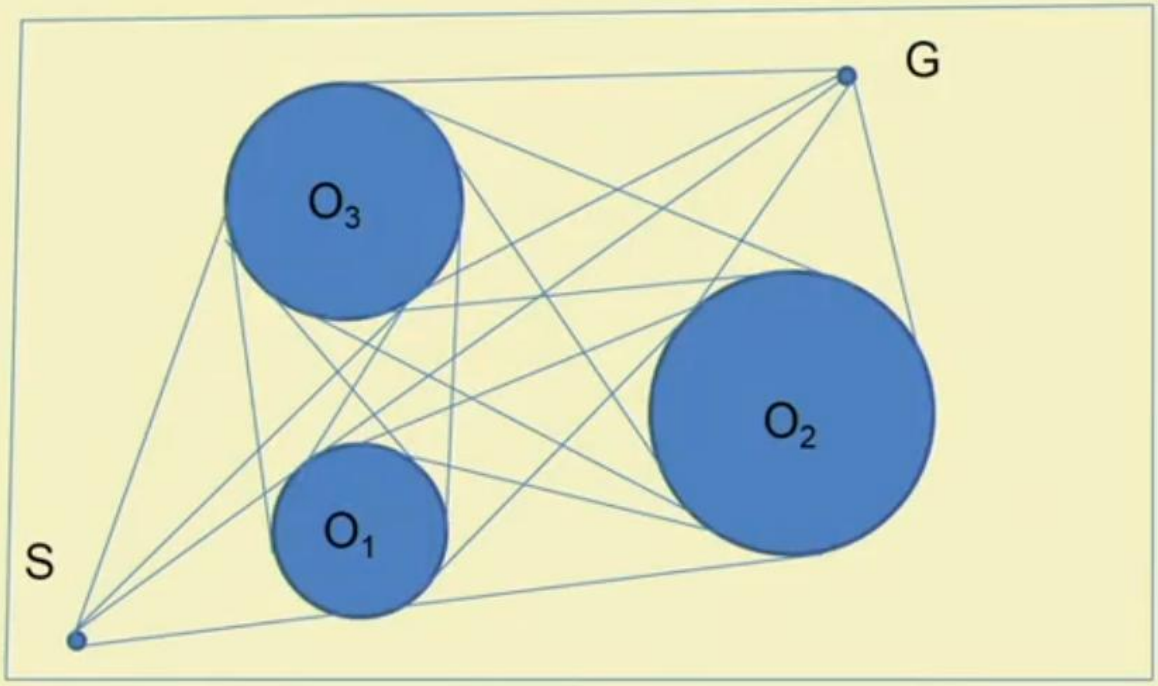
Cell Decomposition (contd.)

- Robot's free area is divided into a number of small regions called cells. A connectivity graph is then constructed and searched.



Tangent Graph (Liu & Arimoto 1991)

- Tangents are drawn from the starting point to the visible obstacle and then from one obstacle to another
- A path comprises of tangents and circular arcs
- Complexity: O (N^2), where N is the number of control points



Approaches to Solve Moving Obstacle Problems

Path Velocity Decomposition (Kant and Zucker, 1984)

This problem is decomposed into two sub-problems as follows:

(i) Path planning problem (PPP) – to plan a path to avoid collision with static obstacles

(ii) Velocity planning problem (VPP) – to plan the velocity of the robot along the above planned path to avoid collision with moving obstacles

Drawbacks

- The path may not be always good, particularly when there are many obstacles
- As there is a sudden change of velocity of the robot, it will have jerky motion, which is not desirable.

Accessibility Graph (Fujimura and samet, 1988)

- Generalization of the visibility graph
- At a particular instant of time, motion planning problem in dynamic environment is converted into find-path problem, which is solved using visibility graph
- In dynamic environment, visibility graph will go on changing.

Incremental Planning Scheme (Slack & Miller, 1987)

Plan a collision – free path based on predicted position of obstacles

↓

Follow the path until it becomes invalid (validity is checked based on the data collected using sensor(s) / camera(s)

↓

Adjust the predictions & plan a new collision – free path

↓

Follow the path until it becomes invalid

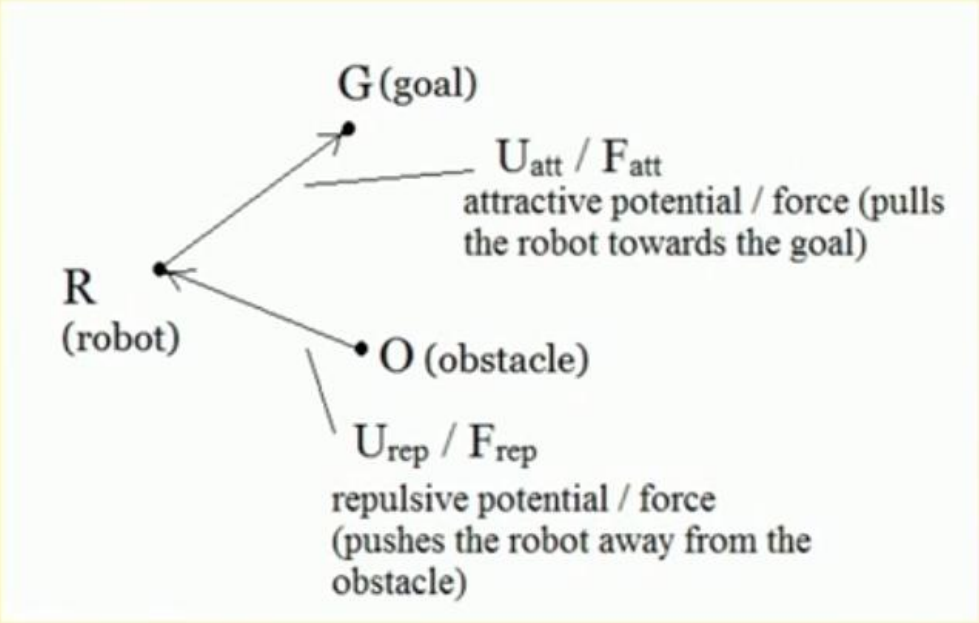
↓

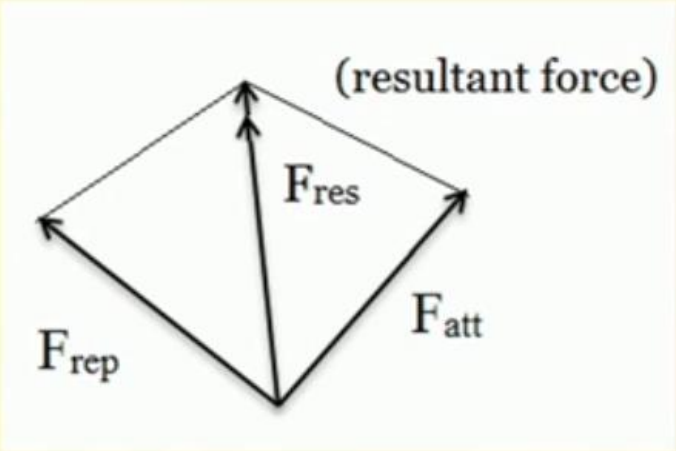
Continue until the robot reaches its destination

Relative Velocity Scheme

- Consider relative velocity of the robot with respect to obstacles
- Dynamic motion planning problem is converted into several static problems
- Several static problems are then converted into a single problem by means of a vector transformation
- Set of velocity vectors are then computed, so that the robot avoids collision with all the moving obstacles

Potential Field Approach (Khatib, 1986)

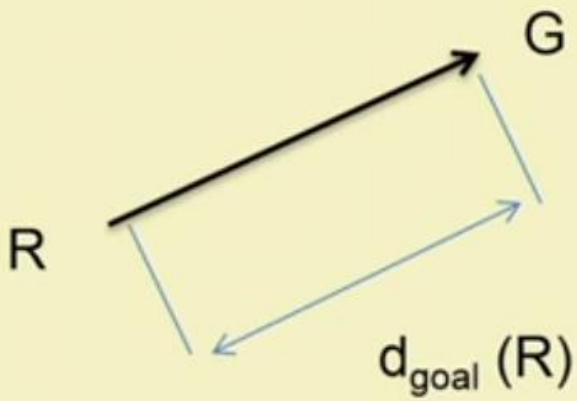


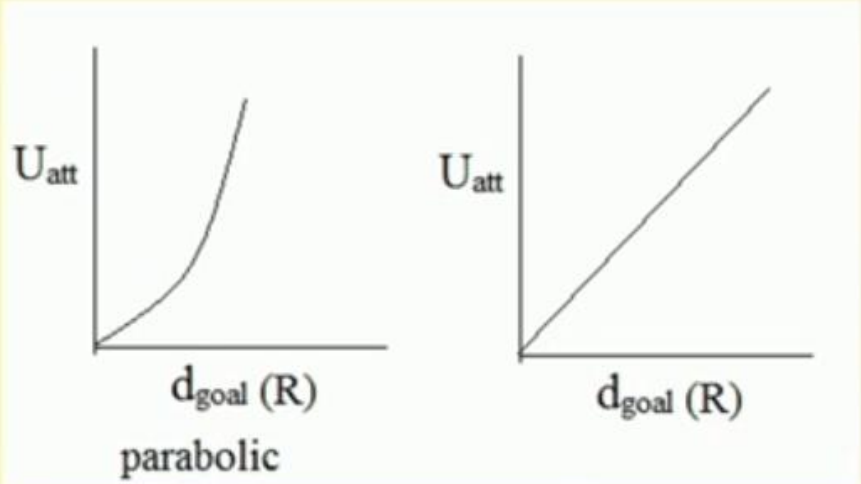


- **Speed of the robot α F_{res}**
- **Direction of movement of the robot is along the direction of resultant force**

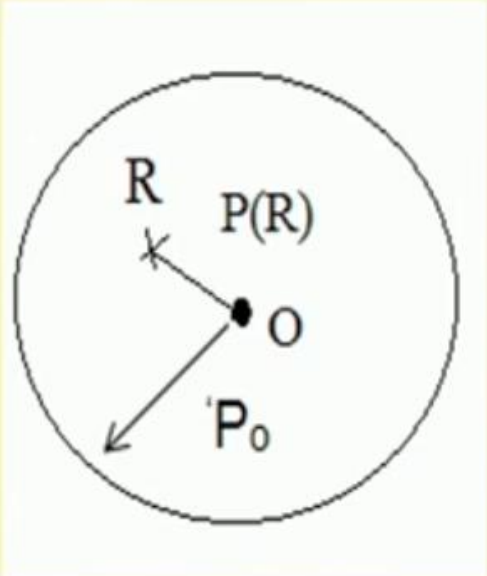
Attractive Potential

$$U_{att}(R) = \frac{1}{2}\xi d_{goal}^2(R), \text{ if parabolic}$$
$$= \xi d_{goal}(R), \text{ if conic-well}$$

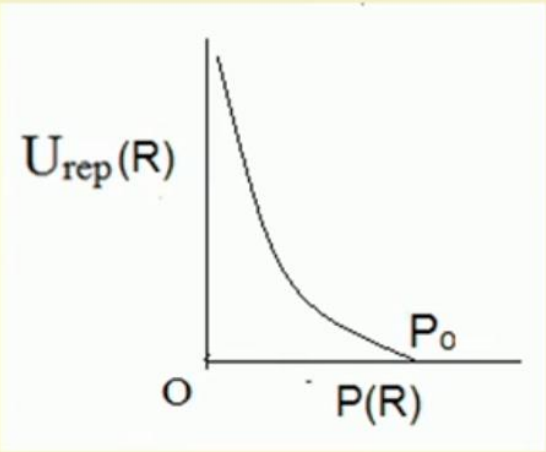




Repulsive Potential



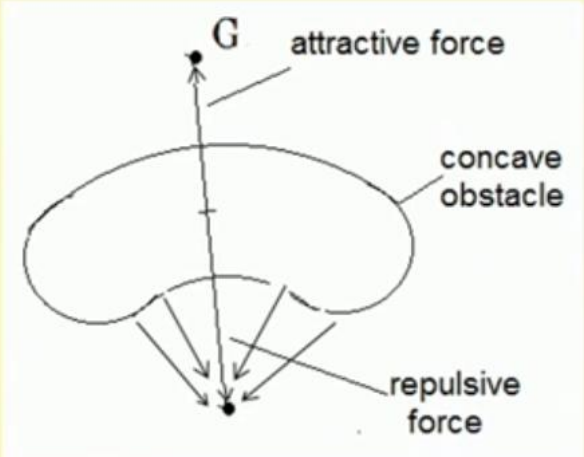
$$U_{rep}(R) = 0$$



$$U_{rep}(R) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{P(R)} - \frac{1}{P_o}\right)^2, & \text{if } P(R) < P_o \\ 0, & \text{if } P(R) \geq P_o \end{cases}$$

Drawbacks

- Solution depends on the chosen potential function
- Chance of local minima problem – when the attractive force is balanced by the repulsive force



- When the robot travels in a narrow corridor, it experiences repulsive forces simultaneously from the opposite sides, and consequently the motion becomes unstable

- Unable to find a path among closely spaced obstacles

Reactive Control Strategy (Brooks, 1986)

- Robotic action is decomposed into some independent primitive behaviours like move-to-goal, avoid-obstacle, etc.
- Basic behaviours are controlled at different layers of control architecture
- Basic behaviours are coordinated by a central mechanism (Behaviour-Based Robotics)

Drawbacks

- The behaviours are hard-wired, thus it is unable to handle behaviours which the programmer did not foresee beforehand
- The number of layers increases with the complexity of the problem. It requires a large amount of computer memory

Computational Complexity

- Canny and Reif (1987) –

  Motion planning for a point robot among moving obstacles in 2D plane with bounded velocity is NP-hard.

- Reif and Sharir (1985) –

  a) Motion planning among moving obstacles in 3-D space without velocity bound is NP-hard

  b) Motion planning among moving obstacles in 3-D space with velocity bound is PSPACE-hard

Drawbacks of the Traditional Methods of Motion Planning

- Traditional methods are computationally expensive even for a simple problem
- No versatile algorithm, which is applicable to all the problems
- As most of the algorithms do not have an optimization module, the generated path may not be optimal in any sense.

So, there is still a need for the development of an efficient, versatile and computationally tractable algorithm for solving the motion planning problems of robots.

Sampling Based Methods

- Sampling-based methods revolutionized robotics by allowing machines to navigate high-dimensional spaces without the need to calculate every possible position on a grid.
- In robotics, **Sample-Based Planning** (often referred to as Sampling-Based Methods) is a paradigm designed to solve the "Piano Mover's Problem" in high-dimensional spaces where traditional grid-based searches like A* become computationally impossible.
- Instead of discretizing the entire environment into a grid, these methods randomly "sample" the **Configuration Space (C-Space)** to find a valid path.
- Two Types: 1. Rapidly-exploring Random Trees (RRT)

  2. Probabilistic Roadmaps (PRM)

Probabilistic Roadmaps (PRM) - 1996

- **The Development:** The PRM was formally introduced in 1996 by **Lydia Kavraki** and her colleagues. It was designed to solve the "Multi-Query" problem, where a robot needs to move between many different points in the same environment (like a warehouse robot).
- **How it Works:** PRM is a two-phase process:

1. **Learning Phase:** It scatters random points (samples) across the map. It keeps the "milestones" that are in free space and connects them to nearby points if the path is clear, creating a "roadmap" (graph).
2. **Query Phase:** To move from A to B, the robot simply finds the closest points on the roadmap and uses a standard search like A* to find the path through the pre-built graph.

- **Example:** Imagine a **robotic arm** in a factory. The environment (the table and cage) never changes. You build a PRM once. Now, whether the robot needs to pick up a bolt or a hammer, it uses the same pre-calculated "roadmap" to move safely.

Rapidly-exploring Random Trees (RRT) - 1998

- **The Development:** As we discussed, Steven M. LaValle developed RRT in 1998 to handle "Single-Query" problems and non-holonomic constraints (robots that can't move in any direction, like a car).
- **How it Works:** Unlike PRM, RRT builds a tree specifically for the current start and goal.
  - It picks a random point (x_{rand}).
  - It finds the node already in the tree closest to that point (x_{near}).
  - It moves a small distance toward the random point to create a new node (x_{new}).
- **Example:** A self-driving car in a parking lot. The "obstacles" (other cars) are always changing. The car doesn't have time to build a full roadmap; it just needs to find one safe way to its parking spot right now. RRT quickly "grows" a path that respects the car's turning radius.

RRT* (Optimal RRT) - 2010

- **The Development:** Developed by **Sertac Karaman** and **Emilio Frazzoli** in 2010. They proved that while RRT is "probabilistically complete" (it will find a path), it is not "optimal" (it won't find the *best* path).
- **How it Works:** RRT* adds two critical steps: **Choose Parent** and **Rewire**.
- When a new node is added, the algorithm checks if it can reach the "start" more efficiently by connecting to a different nearby neighbor.
- If it finds a better way, it "rewires" the tree, constantly shortening the path as it samples more points.
- **Example:** A **delivery drone** flying through a forest. A standard RRT might find a path that zigs and zags wildly, wasting battery. RRT* will constantly refine that path into a smooth, efficient line as it continues to process.

Task Planning

- **Definition:** Task planning is the high-level reasoning process that determines a sequence of discrete actions to achieve a goal.
- **Symbolic Reasoning:** Unlike path planning, which uses coordinates, task planning uses logic and symbols

  (e.g., Robot_At(Table1), Object_Grasped(BlockA)).
- **Purpose:** It bridges the gap between a human-level request (e.g., "Clean the room") and the robot's physical movement.

Hierarchical Planning (HTN)

- Managing Complexity through Abstraction
- Hierarchical Task Networks (HTN): A planning methodology that breaks complex missions into multiple layers of abstraction.
- Structure:
- High-Level Tasks: Abstract goals (e.g., "Assemble Engine").
- Sub-Tasks: Mid-level steps (e.g., "Fetch Piston," "Insert Piston").
- Primitive Tasks: Actions the robot can execute directly (e.g., "MoveTo(x,y)", "CloseGripper").
- Benefit: This hierarchy reduces computational load by narrowing the search space at each level.

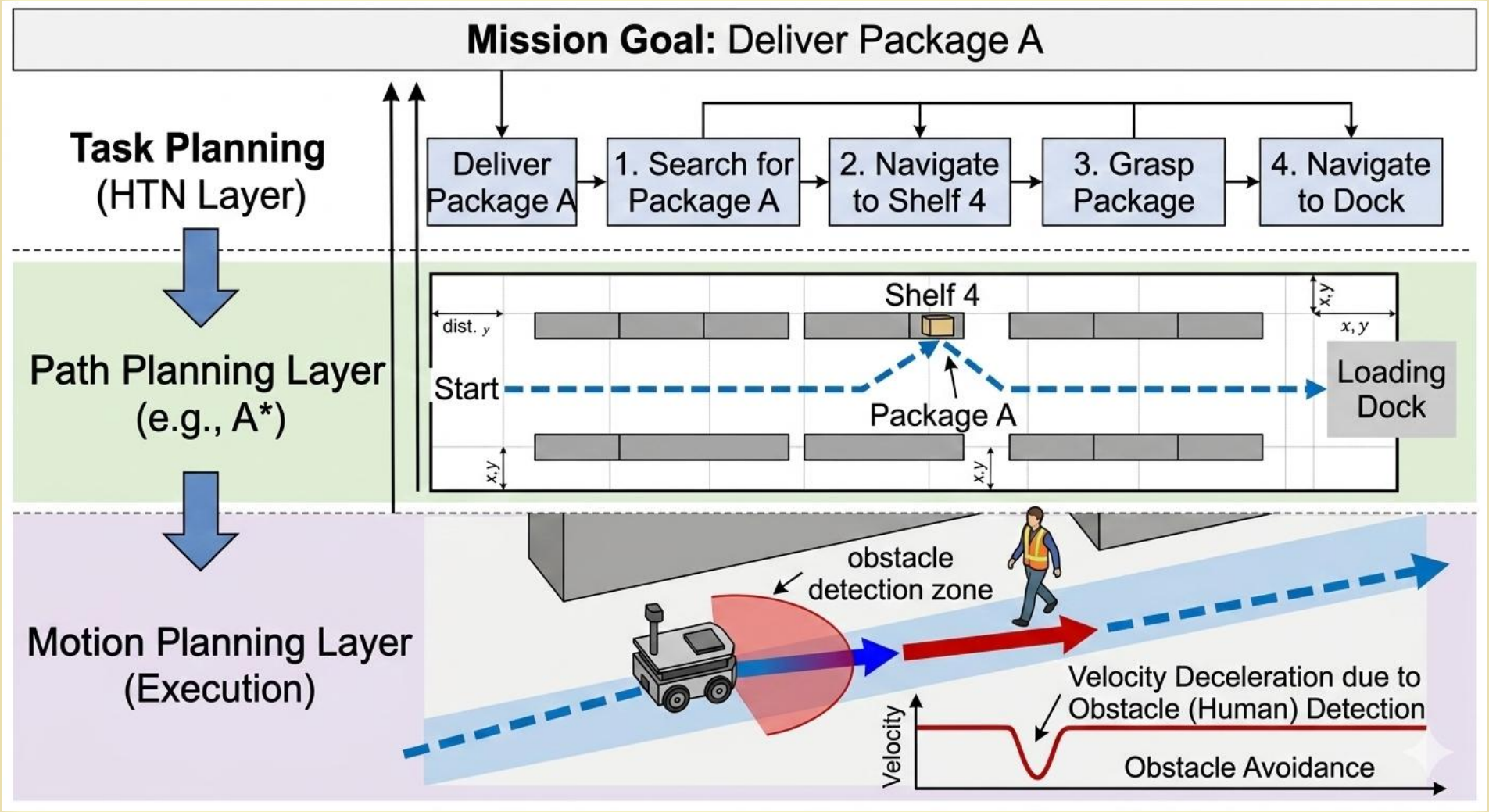
Task Decomposition

- The "How-To" of Planning
- Decomposition Process: The act of breaking a compound task into simpler components until only executable primitive actions remain.
- Methods: One high-level task can have multiple decomposition "methods" depending on the environment (e.g., "Enter Room" can be decomposed into "Open Door" or "Wait for someone to open door").
- Preconditions & Effects:
- Preconditions: Conditions that must be true to start a task (e.g., must be near the object to pick it up).
- Effects: Changes in the world state after task completion (e.g., the robot is now holding the object).

Connecting Task, Path, and Motion Planning

- **Task Planning (Strategic)**: Decides **what** to do (Sequence of logical actions).
- **Path Planning (Tactical)**: Decides **where** to go (Sequence of collision-free waypoints using A* or RRT).
- **Motion Planning (Operational)**: Decides **how** to move (Trajectory generation and obstacle avoidance).
- **Integration**: Task Planning provides the "target" for the Path Planner, which in turn provides the "route" for the Motion Planner.

Case Study


Mission Goal: Deliver Package A
Task Planning
(HTN Layer)
Deliver Package A
1. Search for Package A
2. Navigate to Shelf 4
3. Grasp Package
4. Navigate to Dock
Path Planning Layer
(e.g., A*)
Shelf 4
dist. y
x, y
Start
Package A
Loading Dock
Motion Planning Layer
(Execution)
obstacle detection zone
Velocity Deceleration due to Obstacle (Human) Detection
Velocity
Obstacle Avoidance